Baseline Report - Programmatic Substitutability Judge (PAJAMA adaptation)

A pointwise-binary adaptation of PAJAMA that distills an LLM substitutability judge into a committee of executable Python programs. This reports the baseline run.

1. Input size and curation

The task is a **pointwise binary judge**: given a recipe ingredient and a candidate USDA food entry, decide whether the entry is a *substitutable* match - usable in place of the ingredient without changing preparation and without a diner noticing a difference in taste, texture, or appearance.

The dataset is **943 labeled rows**, each `(ingredient, usda_match, substitutablein{0,1})`, with class balance **671 substitutable / 272 not (71% positive)**. Labels were produced upstream by an LLM substitutability judge over recipe->USDA matches and are treated as the gold target to reproduce. Rows were deduplicated and divided by a **grouped, label-stratified split** keyed on the gram-stripped `(ingredient, usda_match)` pair, so identical program inputs never straddle splits (verified **zero cross-split overlap**). This yields **val = 222** and **test = 721**. From val, **10 balanced few-shot examples** (~5 positive / ~5 negative) were sampled to ground program synthesis - only ~1% of the data is ever shown to the generator.

2. LLM configuration (synthesis only)

The LLM is invoked **once per program at synthesis time**, never at scoring time.

Setting	Value
Model	`claude-opus-4-6`
Temperature	0.7
max_tokens	5000
Extended thinking	off
Calls	**80** (10 rubrics x 8 variants), each independent
Per call	~1.3k input / ~2k output tokens
Total tokens	~0.1M input + ~0.16M output
Estimated cost	**~ $4.5** ($4-8 with validation/similarity retries)

Each call receives task instructions + one evaluation rubric + the 10 few-shot examples and returns one `judging_function(ingredient, usda_match)`. After synthesis, all scoring runs locally at **zero marginal API cost**.

3. What the programs look like

Every program is a self-contained `judging_function(ingredient, usda_match) -> float` (higher = more substitutable), using only the standard library (`re`, `collections`, `math`), wrapped in `try/except`

with a numeric fallback. They parse the two strings - stripping gram prefixes (`300g`), parentheticals, and noise words - then score with lightweight string/token features and penalties.

Deterministic baselines (4 hand-written reference programs): - `judge_1` - Jaccard overlap of meaningful food tokens. - `judge_2` - core-food *head* match (ingredient head noun vs the USDA primary food, i.e. text before the first comma). - `judge_3` - processing/additive penalty (downweights entries that add *sauce / paste / sweetened / seasoning / cooked*). - `judge_4` - character-trigram cosine similarity (fuzzy spelling/variant match).

Top selected synthesized programs (the committee's top entries by validation accuracy; each realizes a different rubric with a distinct algorithm):

Program	Rubric	Approach
`judge_54`	Food Category Consistency	Classifies both strings into food categories (dairy, meat, poultry, fish, shellfish, ...) via keyword sets; scores same-category, penalizes cross-category.
`judge_11`	Preparation-State Equivalence	Separates *trivial* prep states (raw, chopped, sliced) from *significant* ones (cooked, fried); Jaccard on core food words with penalties for significant-state mismatch.
`judge_19`	Preservation & Form Match	Buckets form keywords (frozen / canned / dried / fresh / jarred / paste); compatibility matrix penalizing water-content/handling changes.
`judge_6`	Core Food Identity Match	Cleans to core text, builds token `Counters`, cosine-style overlap on the primary food identity.
`judge_9`	Preparation-State Equivalence	Trivial-state whitelist + core-token overlap; a different variant of the prep-state logic than `judge_11`.
`judge_41`	Lexical & Token Overlap	Noise-word-filtered token extraction, then set/overlap scoring of the food-defining tokens.

The transparency is the point: each line can be read, audited, and edited - a verdict traces to explicit features, unlike an opaque LLM judge.

4. Evaluation - how measured

Programs are fit on **val only**: raw scores are min-max normalized (robust P1/P99), a per-program decision threshold theta is grid-searched for maximum covered-row accuracy, programs below 0.50 val accuracy are dropped, a **top-k committee** is chosen by val agreement, and votes are fused with a **Snorkel `LabelModel` (cardinality=2)**. The fitted committee is then applied to the **held-out 721-row test set**. Metrics: **accuracy** (agreement with the LLM judge's label), **precision/recall/F1** for the positive (substitutable) class, and **coverage** (fraction of non-abstained rows), with a **majority-class baseline** for reference.

Results (held-out test), with PAJAMA paper figures for reference:

System	Task	Accuracy	F1
This work - 80 synthesized programs	Pointwise binary substitutability (USDA)	**0.749**	**0.824**
PAJAMA - average of 5 datasets (paper)	Pairwise preference	0.781	n/r
PAJAMA - Prometheus, its best (paper)	Pairwise preference	0.888	n/r
PAJAMA - JudgeLM (paper)	Pairwise preference	0.811	n/r
OLMo-2-13B-Instruct LLM judge (paper ref)	Pairwise preference	0.797	n/r
GPT-5 Thinking, strongest judge (paper)	Pairwise preference	0.857	n/r

Our run also reported precision 0.803, recall 0.846, coverage 1.0, and **val agreement 0.955**.

> **n/r = not reported.** PAJAMA's headline metrics are **accuracy** and **throughput** (samples/sec) - see paper Table 4 - not F1; it does not publish F1 anywhere. The paper's tasks are **pairwise preference** (which of two responses is better), whereas ours is **pointwise binary** (where F1 on the positive class is meaningful), so the rows are not a head-to-head comparison - the paper figures simply mark the accuracy regime PAJAMA operates in. On its own tasks PAJAMA averages **78.1%** accuracy, matching a mid-size (OLMo-2-13B) LLM judge and trailing the strongest judge (GPT-5 Thinking, 85.7%) by ~8 pts.

5. Analysis and how PAJAMA could be improved

The committee reproduces the LLM judge at **0.749 accuracy / 0.824 F1**, at full coverage and near-zero inference cost - the core PAJAMA value proposition (local, cheap, auditable scoring) holds, and the accuracy sits squarely in the regime PAJAMA reports on its own pairwise tasks (~0.70-0.89 per dataset). Two signals stand out. First, the **0.955 val -> 0.749 test gap** indicates **overfitting**: with only 222 val rows, `best_k` selection by val agreement latches onto validation noise. Second, in internal checks a handful of simple hand-written heuristics scored comparably, suggesting the 80 synthesized programs add variance faster than signal on this task.

Concretely, PAJAMA could be improved by: 1. **Larger validation set** and `best_k` capped near the ~20 plateau to stabilize selection. 2. Stronger **diversity / similarity filtering** and pruning of redundant rubric variants. 3. **Class-imbalance-aware** thresholding and metrics, given the 71% positive skew. 4. Adding the paper's **confidence-gated LLM fallback** (absent in the released pipeline) to escalate low-margin rows to an LLM judge. 5. **Program calibration** - editing the transparent program logic to fix systematic errors, the auditability advantage programs hold over an opaque LLM judge.